SYNAPSE DevOps & Deployment Guide

AI-Powered Technology Intelligence Platform

DevOps Engineering Team

March 7, 2026

Contents

1 Executive Summary **2**
1.1 Technology Stack Overview . . . 2

2 DevOps Philosophy & Principles **3**
2.1 Core Principles . . . 3
2.1.1 Infrastructure as Code (IaC) . . . 3
2.1.2 CI/CD-First Development . . . 3
2.1.3 Container-First Deployment . . . 3
2.1.4 Immutable Infrastructure . . . 3
2.1.5 12-Factor App Methodology . . . 4
2.1.6 Zero-Downtime Deployments . . . 4

3 Local Development Environment **5**
3.1 Overview . . . 5
3.2 Docker Compose Configuration . . . 5
3.3 Environment Variables Management . . . 8
3.3.1 Local Development (.env.local) . . . 8
3.3.2 Production Environment (.env.production) . . . 9
3.4 Getting Started . . . 10

4 Dockerfile Specifications **11**
4.1 Backend Dockerfile (Django + Gunicorn) . . . 11
4.2 FastAPI Dockerfile (Uvicorn) . . . 12
4.3 Frontend Dockerfile (Next.js) . . . 13
4.4 Celery Worker Dockerfile . . . 14
4.5 .dockerignore Files . . . 14

5 CI/CD Pipeline (GitHub Actions) **16**
5.1 Overview . . . 16
5.2 Continuous Integration Workflow . . . 16
5.3 Continuous Deployment Workflow . . . 20
5.4 GitHub Actions Best Practices . . . 23
5.4.1 Secrets Management . . . 23
5.4.2 Caching Strategy . . . 23

6 Production Infrastructure (AWS) **24**
6.1 Architecture Overview . . . 24
6.2 EC2 Instance Recommendations . . . 24
6.2.1 Backend Service . . . 24
6.2.2 FastAPI AI Service . . . 25
6.3 RDS PostgreSQL Configuration . . . 25
6.4 ElastiCache Redis Configuration . . . 25
6.5 Auto-Scaling Configuration . . . 26

7 Nginx Configuration **27**
7.1 Reverse Proxy Setup . . . 27

8 Environment Configuration **31**
8.1 Required Environment Variables . . . 31
8.2 Secrets Management Strategy . . . 32
8.2.1 AWS Secrets Manager . . . 32
8.2.2 EC2 IAM Role . . . 32
8.2.3 Deployment Secret Injection . . . 33

9 Database Management in Production **34**
9.1 Database Migrations Strategy . . . 34
9.2 Backup Strategy . . . 34
9.2.1 Automated Daily Backups . . . 34
9.2.2 Point-in-Time Recovery . . . 35
9.3 Read Replicas for Reporting . . . 35
9.4 pgvector Index Maintenance . . . 35
9.4.1 Index Creation . . . 35
9.4.2 Index Maintenance . . . 35
9.5 Redis Persistence . . . 36

10 Monitoring & Observability **37**
10.1 Prometheus Metrics Setup . . . 37
10.1.1 Django Metrics with django-prometheus . . . 37
10.1.2 Custom Business Metrics . . . 37
10.2 Grafana Dashboards . . . 38
10.2.1 System Overview Dashboard . . . 38
10.2.2 Application Metrics Dashboard . . . 38
10.2.3 Business Metrics Dashboard . . . 39
10.2.4 Celery Task Monitoring . . . 39
10.3 Sentry Integration . . . 39
10.3.1 Django Setup . . . 39
10.3.2 Next.js Setup . . . 39
10.4 Structured Logging . . . 40
10.4.1 Log Aggregation with CloudWatch . . . 40
10.5 Uptime Monitoring . . . 41
10.5.1 AWS Route 53 Health Checks . . . 41
10.6 Alert Rules . . . 41

11 Security Hardening **43**
11.1 Django Security Settings . . . 43
11.2 Database Security . . . 44
11.3 Docker Security . . . 44
11.4 Network Security . . . 44
11.5 Dependency Scanning . . . 45
11.5.1 Python Dependencies . . . 45
11.5.2 Node.js Dependencies . . . 45
11.6 Secret Rotation Strategy . . . 45

12 Scaling Strategy **46**
12.1 Horizontal Scaling of Backend Services . . . 46
12.2 Celery Worker Scaling . . . 46
12.3 Database Connection Pooling (PgBouncer) . . . 47
12.4 CDN for Static Assets . . . 47
12.5 Redis Cluster for High Availability . . . 47

12.6 Load Testing . . . 47

13 Disaster Recovery **49**
13.1 Recovery Objectives . . . 49
13.2 Multi-AZ Database Deployment . . . 49
13.3 S3 Versioning and Cross-Region Replication . . . 49
13.4 Infrastructure Replication . . . 50
13.5 Disaster Recovery Runbook . . . 50
13.5.1 Database Corruption . . . 50
13.5.2 Complete Data Center Failure . . . 50
13.5.3 Ransomware/Security Incident . . . 51

14 Deployment Runbook **52**
14.1 Pre-Deployment Checklist . . . 52
14.2 Step-by-Step Deployment Procedure . . . 52
14.2.1 Manual Deployment (if CI/CD unavailable) . . . 53
14.3 Rollback Procedure . . . 54
14.4 Health Check Endpoints . . . 54
14.5 Post-Deployment Verification Checklist . . . 55
14.6 Maintenance Window Procedure . . . 55

A Useful Commands **58**
A.1 Docker Compose Commands . . . 58
A.2 AWS CLI Commands . . . 58
A.3 Database Commands . . . 59

Chapter 1

Executive Summary

SYNAPSE is a FAANG-style AI-powered technology intelligence and automation platform built with a modern, cloud-native DevOps architecture. This document provides comprehensive guidance on deploying, managing, and scaling SYNAPSE in development, staging, and production environments.

1.1 Technology Stack Overview

- **Backend Framework**: Django 4.2 + FastAPI 0.104 (Python 3.11)
- **Frontend Framework**: Next.js 14
- **Databases**: PostgreSQL 15, Redis 7, pgvector
- **Message Queue**: Celery 5.3
- **Containerization**: Docker, Docker Compose
- **CI/CD**: GitHub Actions
- **Cloud Platforms**: AWS / GCP
- **Monitoring**: Prometheus, Grafana, Sentry
- **Reverse Proxy**: Nginx

Chapter 2

DevOps Philosophy & Principles

2.1 Core Principles

SYNAPSE deployment follows industry-standard DevOps best practices designed for reliability, scalability, and maintainability:

2.1.1 Infrastructure as Code (IaC)

All infrastructure is defined, versioned, and automated through code. Docker Compose for local development and cloud infrastructure templates (Terraform/CloudFormation) for production ensure reproducibility and consistency across environments. No manual infrastructure changes are permitted in production.

2.1.2 CI/CD-First Development

Every code change triggers an automated pipeline:

1. Linting and code quality checks
2. Automated testing (unit, integration, end-to-end)
3. Security scanning (SAST, dependency scanning)
4. Container image building and pushing
5. Automated deployment to staging
6. Manual promotion to production with health checks

2.1.3 Container-First Deployment

All services run in Docker containers with consistent, reproducible builds. This eliminates "works on my machine" problems and simplifies deployment across different environments.

2.1.4 Immutable Infrastructure

Docker images are built once and deployed to multiple environments without modification. Configuration is injected at runtime via environment variables. This ensures production environments match tested staging environments exactly.

2.1.5 12-Factor App Methodology

SYNAPSE adheres to the twelve-factor app principles:

- Codebase tracked in git
- Dependencies explicitly declared in requirements.txt and package.json
- Configuration via environment variables
- Backing services (databases, cache) as attached resources
- Strict separation of build, release, and run stages
- Stateless processes
- Port binding (services export HTTP as a port-bound service)
- Concurrency through process types
- Fast startup and graceful shutdown
- Feature parity between dev/staging/production
- Logs as event streams
- Admin tasks as one-off processes

2.1.6 Zero-Downtime Deployments

Production deployments use rolling updates with health checks. Services gracefully handle SIGTERM signals, complete in-flight requests, and shut down cleanly. Load balancers drain connections before removing instances.

Chapter 3

Local Development Environment

3.1 Overview

The local development environment mirrors production as closely as possible using Docker Compose. This ensures consistency and reduces environment-specific bugs.

3.2 Docker Compose Configuration

Below is the complete `docker-compose.yml` file for local development:

```yaml
version: '3.9'

services:
  # PostgreSQL Database
  postgres:
    image: postgres:15-alpine
    container_name: synapse_postgres
    environment:
      POSTGRES_DB: synapse_db
      POSTGRES_USER: synapse_user
      POSTGRES_PASSWORD: synapse_password
    ports:
      - "5432:5432"
    volumes:
      - postgres_data:/var/lib/postgresql/data
      - ./scripts/init-db.sql:/docker-entrypoint-initdb.d/init.sql
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U synapse_user -d synapse_db"]
      interval: 10s
      timeout: 5s
      retries: 5
    networks:
      - synapse_network

  # Redis Cache
  redis:
    image: redis:7-alpine
    container_name: synapse_redis
    ports:
      - "6379:6379"
    volumes:
      - redis_data:/data
```

```yaml
    command: redis-server --appendonly yes
    healthcheck:
      test: ["CMD", "redis-cli", "ping"]
      interval: 10s
      timeout: 5s
      retries: 5
    networks:
      - synapse_network

  # Django Backend Service
  backend:
    build:
      context: ./backend
      dockerfile: Dockerfile.dev
    container_name: synapse_backend
    command: python manage.py runserver 0.0.0.0:8000
    environment:
      DEBUG: "True"
      DJANGO_SETTINGS_MODULE: synapse.settings.development
      DATABASE_URL:
          postgresql://synapse_user:synapse_password@postgres:5432/synapse_db
      REDIS_URL: redis://redis:6379/0
      SECRET_KEY: dev-secret-key-change-in-production
      ALLOWED_HOSTS: localhost,127.0.0.1,backend
      CELERY_BROKER_URL: redis://redis:6379/0
      CELERY_RESULT_BACKEND: redis://redis:6379/0
    ports:
      - "8000:8000"
    volumes:
      - ./backend:/app
      - /app/.venv
    depends_on:
      postgres:
        condition: service_healthy
      redis:
        condition: service_healthy
    networks:
      - synapse_network

  # FastAPI AI Service
  fastapi_ai:
    build:
      context: ./fastapi_ai
      dockerfile: Dockerfile.dev
    container_name: synapse_fastapi_ai
    command: uvicorn main:app --host 0.0.0.0 --port 8001 --reload
    environment:
      DEBUG: "True"
      DATABASE_URL:
          postgresql://synapse_user:synapse_password@postgres:5432/synapse_db
      REDIS_URL: redis://redis:6379/1
      OPENAI_API_KEY: ${OPENAI_API_KEY:-sk-test-key}
      PINECONE_API_KEY: ${PINECONE_API_KEY:-test-key}
      PINECONE_ENV: ${PINECONE_ENV:-gcp-starter}
    ports:
      - "8001:8001"
    volumes:
      - ./fastapi_ai:/app
      - /app/.venv
```

```yaml
    depends_on:
      postgres:
        condition: service_healthy
      redis:
        condition: service_healthy
    networks:
      - synapse_network

  # Next.js Frontend
  frontend:
    build:
      context: ./frontend
      dockerfile: Dockerfile.dev
    container_name: synapse_frontend
    command: npm run dev
    environment:
      NEXT_PUBLIC_API_URL: http://localhost:8000
      NEXT_PUBLIC_FASTAPI_URL: http://localhost:8001
      NEXT_PUBLIC_WS_URL: ws://localhost:8000/ws
    ports:
      - "3000:3000"
    volumes:
      - ./frontend:/app
      - /app/node_modules
      - /app/.next
    depends_on:
      - backend
      - fastapi_ai
    networks:
      - synapse_network

  # Celery Worker
  celery_worker:
    build:
      context: ./backend
      dockerfile: Dockerfile.dev
    container_name: synapse_celery_worker
    command: celery -A synapse worker -l info
    environment:
      DEBUG: "True"
      DJANGO_SETTINGS_MODULE: synapse.settings.development
      DATABASE_URL:
          postgresql://synapse_user:synapse_password@postgres:5432/synapse_db
      REDIS_URL: redis://redis:6379/0
      CELERY_BROKER_URL: redis://redis:6379/0
      CELERY_RESULT_BACKEND: redis://redis:6379/0
    volumes:
      - ./backend:/app
      - /app/.venv
    depends_on:
      postgres:
        condition: service_healthy
      redis:
        condition: service_healthy
    networks:
      - synapse_network

  # Celery Beat Scheduler
  celery_beat:
```

```yaml
    build:
      context: ./backend
      dockerfile: Dockerfile.dev
    container_name: synapse_celery_beat
    command: celery -A synapse beat -l info --scheduler
        django_celery_beat.schedulers:DatabaseScheduler
    environment:
      DEBUG: "True"
      DJANGO_SETTINGS_MODULE: synapse.settings.development
      DATABASE_URL:
          postgresql://synapse_user:synapse_password@postgres:5432/synapse_db
      REDIS_URL: redis://redis:6379/0
      CELERY_BROKER_URL: redis://redis:6379/0
      CELERY_RESULT_BACKEND: redis://redis:6379/0
    volumes:
      - ./backend:/app
      - /app/.venv
    depends_on:
      postgres:
        condition: service_healthy
      redis:
        condition: service_healthy
    networks:
      - synapse_network

  # PgAdmin (Optional)
  pgadmin:
    image: dpage/pgadmin4:latest
    container_name: synapse_pgadmin
    environment:
      PGADMIN_DEFAULT_EMAIL: admin@synapse.local
      PGADMIN_DEFAULT_PASSWORD: pgadmin_password
    ports:
      - "5050:80"
    depends_on:
      - postgres
    networks:
      - synapse_network

volumes:
  postgres_data:
  redis_data:

networks:
  synapse_network:
    driver: bridge
```

Listing 3.1: docker-compose.yml - Local Development Stack

3.3 Environment Variables Management

3.3.1 Local Development (.env.local)

Create `.env.local` in the project root:

```python
# Django Settings
DEBUG=True
SECRET_KEY=dev-secret-key-change-in-production
```

```
ALLOWED_HOSTS=localhost,127.0.0.1,backend,frontend

# Database
DATABASE_URL=postgresql://synapse_user:synapse_password@postgres:5432/synapse_db
POSTGRES_DB=synapse_db
POSTGRES_USER=synapse_user
POSTGRES_PASSWORD=synapse_password

# Redis
REDIS_URL=redis://redis:6379/0

# Celery
CELERY_BROKER_URL=redis://redis:6379/0
CELERY_RESULT_BACKEND=redis://redis:6379/0

# AI/ML Services
OPENAI_API_KEY=sk-test-key
PINECONE_API_KEY=test-key
PINECONE_ENV=gcp-starter

# External APIs
GITHUB_TOKEN=ghp_test_token
ARXIV_API_KEY=test-arxiv-key
YOUTUBE_API_KEY=test-youtube-key
NEWS_API_KEY=test-news-key

# Email
SENDGRID_API_KEY=test-sendgrid-key

# Frontend
NEXT_PUBLIC_API_URL=http://localhost:8000
NEXT_PUBLIC_FASTAPI_URL=http://localhost:8001
NEXT_PUBLIC_WS_URL=ws://localhost:8000/ws
```

Listing 3.2: .env.local - Local Development Environment Variables

3.3.2 Production Environment (.env.production)

```
# Django Settings
DEBUG=False
SECRET_KEY=${DJANGO_SECRET_KEY}
ALLOWED_HOSTS=synapse.example.com,api.synapse.example.com
CSRF_TRUSTED_ORIGINS=https://synapse.example.com,https://api.synapse.example.com

# Security
SECURE_SSL_REDIRECT=True
SESSION_COOKIE_SECURE=True
CSRF_COOKIE_SECURE=True
SECURE_HSTS_SECONDS=31536000
SECURE_HSTS_INCLUDE_SUBDOMAINS=True
SECURE_HSTS_PRELOAD=True

# Database (RDS)
DATABASE_URL=postgresql://${DB_USER}:${DB_PASSWORD}@${DB_HOST}:5432/${DB_NAME}

# Redis (ElastiCache)
REDIS_URL=redis://${REDIS_HOST}:6379/0
```

```
# Celery
CELERY_BROKER_URL=redis://${REDIS_HOST}:6379/0
CELERY_RESULT_BACKEND=redis://${REDIS_HOST}:6379/0

# AWS
AWS_ACCESS_KEY_ID=${AWS_ACCESS_KEY_ID}
AWS_SECRET_ACCESS_KEY=${AWS_SECRET_ACCESS_KEY}
AWS_STORAGE_BUCKET_NAME=synapse-prod-storage
AWS_S3_REGION_NAME=us-east-1

# AI/ML Services
OPENAI_API_KEY=${OPENAI_API_KEY}
PINECONE_API_KEY=${PINECONE_API_KEY}
PINECONE_ENV=${PINECONE_ENV}

# Monitoring
SENTRY_DSN=${SENTRY_DSN}

# Frontend
NEXT_PUBLIC_API_URL=https://api.synapse.example.com
NEXT_PUBLIC_FASTAPI_URL=https://ai.synapse.example.com
NEXT_PUBLIC_WS_URL=wss://api.synapse.example.com/ws
```

Listing 3.3: .env.production - Production Environment Variables

3.4 Getting Started

To set up the local development environment:

```
# Clone repository
git clone https://github.com/yourorg/synapse.git
cd synapse

# Create .env.local
cp .env.example .env.local
# Edit .env.local with your values

# Build and start all services
docker-compose up -d

# Run migrations
docker-compose exec backend python manage.py migrate

# Create superuser
docker-compose exec backend python manage.py createsuperuser

# View logs
docker-compose logs -f

# Stop services
docker-compose down
```

Listing 3.4: Local Development Setup Commands

Chapter 4

Dockerfile Specifications

4.1 Backend Dockerfile (Django + Gunicorn)

The backend uses a multi-stage build to minimize image size and improve security:

```
# Stage 1: Builder
FROM python:3.11-slim as builder

WORKDIR /app

# Install system dependencies
RUN apt-get update && apt-get install -y --no-install-recommends \
    build-essential \
    libpq-dev \
    && rm -rf /var/lib/apt/lists/*

# Install Python dependencies
COPY requirements.txt .
RUN pip install --user --no-cache-dir -r requirements.txt

# Stage 2: Production
FROM python:3.11-slim

WORKDIR /app

# Install only runtime dependencies
RUN apt-get update && apt-get install -y --no-install-recommends \
    libpq5 \
    postgresql-client \
    && rm -rf /var/lib/apt/lists/*

# Create non-root user for security
RUN useradd -m -u 1000 synapse_user

# Copy Python dependencies from builder
COPY --from=builder /root/.local /home/synapse_user/.local
ENV PATH=/home/synapse_user/.local/bin:$PATH

# Copy application code
COPY --chown=synapse_user:synapse_user . .

# Switch to non-root user
USER synapse_user
```

```
# Health check
HEALTHCHECK --interval=30s --timeout=10s --start-period=40s --retries=3 \
    CMD curl -f http://localhost:8000/health/ || exit 1

# Expose port
EXPOSE 8000

# Run Gunicorn
CMD ["gunicorn", \
     "--bind", "0.0.0.0:8000", \
     "--workers", "4", \
     "--worker-class", "gthread", \
     "--threads", "2", \
     "--worker-tmp-dir", "/dev/shm", \
     "--max-requests", "1000", \
     "--max-requests-jitter", "50", \
     "--timeout", "30", \
     "--access-logfile", "-", \
     "--error-logfile", "-", \
     "synapse.wsgi:application"]
```

Listing 4.1: Dockerfile - Django Backend

4.2 FastAPI Dockerfile (Uvicorn)

```
# Stage 1: Builder
FROM python:3.11-slim as builder

WORKDIR /app

RUN apt-get update && apt-get install -y --no-install-recommends \
    build-essential \
    libpq-dev \
    && rm -rf /var/lib/apt/lists/*

COPY requirements.txt .
RUN pip install --user --no-cache-dir -r requirements.txt

# Stage 2: Production
FROM python:3.11-slim

WORKDIR /app

RUN apt-get update && apt-get install -y --no-install-recommends \
    libpq5 \
    postgresql-client \
    && rm -rf /var/lib/apt/lists/*

RUN useradd -m -u 1000 synapse_user

COPY --from=builder /root/.local /home/synapse_user/.local
ENV PATH=/home/synapse_user/.local/bin:$PATH

COPY --chown=synapse_user:synapse_user . .

USER synapse_user
```

```
HEALTHCHECK --interval=30s --timeout=10s --start-period=40s --retries=3 \
    CMD curl -f http://localhost:8001/health || exit 1

EXPOSE 8001

CMD ["uvicorn", \
     "main:app", \
     "--host", "0.0.0.0", \
     "--port", "8001", \
     "--workers", "4"]
```

Listing 4.2: Dockerfile - FastAPI AI Service

4.3 Frontend Dockerfile (Next.js)

```
# Stage 1: Dependencies
FROM node:20-alpine as deps
WORKDIR /app
COPY package.json package-lock.json ./
RUN npm ci

# Stage 2: Builder
FROM node:20-alpine as builder
WORKDIR /app
COPY --from=deps /app/node_modules ./node_modules
COPY . .
ENV NEXT_TELEMETRY_DISABLED=1
RUN npm run build

# Stage 3: Runner
FROM node:20-alpine as runner
WORKDIR /app

RUN addgroup -g 1001 -S nodejs
RUN adduser -S nextjs -u 1001

COPY --from=builder /app/public ./public
COPY --from=builder /app/.next/standalone ./
COPY --from=builder /app/.next/static ./.next/static

USER nextjs

EXPOSE 3000

ENV NODE_ENV=production
ENV NEXT_TELEMETRY_DISABLED=1

HEALTHCHECK --interval=30s --timeout=10s --start-period=40s --retries=3 \
    CMD node -e "require('http').get('http://localhost:3000/api/health', (r) =>
        {if (r.statusCode !== 200) throw new Error(r.statusCode)})"

CMD ["node", "server.js"]
```

Listing 4.3: Dockerfile - Next.js Frontend

4.4 Celery Worker Dockerfile

The Celery worker uses the same base as the backend but with different entrypoint:

```
FROM python:3.11-slim as builder

WORKDIR /app

RUN apt-get update && apt-get install -y --no-install-recommends \
    build-essential \
    libpq-dev \
    && rm -rf /var/lib/apt/lists/*

COPY requirements.txt .
RUN pip install --user --no-cache-dir -r requirements.txt

FROM python:3.11-slim

WORKDIR /app

RUN apt-get update && apt-get install -y --no-install-recommends \
    libpq5 \
    postgresql-client \
    && rm -rf /var/lib/apt/lists/*

RUN useradd -m -u 1000 synapse_user

COPY --from=builder /root/.local /home/synapse_user/.local
ENV PATH=/home/synapse_user/.local/bin:$PATH

COPY --chown=synapse_user:synapse_user . .

USER synapse_user

CMD ["celery", "-A", "synapse", "worker", "-l", "info", \
     "-c", "4", \
     "--max-tasks-per-child", "100"]
```

Listing 4.4: Dockerfile - Celery Worker

4.5 .dockerignore Files

To reduce image size and build context, create `.dockerignore` files:

```
*.pyc
__pycache__
*.pyo
*.pyd
.Python
env/
venv/
.venv
pip-log.txt
pip-delete-this-directory.txt
.tox/
.coverage
.coverage.*
.cache
```

```
nosetests.xml
coverage.xml
*.cover
*.log
.git
.gitignore
.dockerignore
Dockerfile
docker-compose*.yml
.env*
.pytest_cache/
.mypy_cache/
.vscode/
.idea/
*.swp
*.swo
*~
.DS_Store
```

Listing 4.5: .dockerignore - Backend

```
node_modules
npm-debug.log
build
dist
.next
.env*
.git
.gitignore
.dockerignore
.vscode
.idea
*.swp
*.swo
*~
.DS_Store
.eslintcache
coverage
```

Listing 4.6: .dockerignore - Frontend

Chapter 5

CI/CD Pipeline (GitHub Actions)

5.1 Overview

SYNAPSE uses GitHub Actions for continuous integration and deployment. The pipeline ensures code quality, security, and automated deployment to production.

5.2 Continuous Integration Workflow

```yaml
name: CI

on:
  push:
    branches: [ main, develop ]
  pull_request:
    branches: [ main, develop ]

env:
  REGISTRY: ghcr.io
  IMAGE_NAME: ${{ github.repository }}

jobs:
  lint:
    runs-on: ubuntu-latest
    name: Lint Code

    steps:
      - uses: actions/checkout@v3

      - name: Set up Python
        uses: actions/setup-python@v4
        with:
          python-version: '3.11'
          cache: 'pip'

      - name: Install linting tools
        run: |
          pip install flake8 black isort pylint

      - name: Run flake8
        run: flake8 backend/ --count --select=E9,F63,F7,F82 --show-source
            --statistics
```

```yaml
      - name: Check code formatting with black
        run: black --check backend/

      - name: Check import sorting with isort
        run: isort --check-only backend/

      - name: Set up Node.js
        uses: actions/setup-node@v3
        with:
          node-version: '20'
          cache: 'npm'
          cache-dependency-path: frontend/package-lock.json

      - name: Install ESLint
        working-directory: frontend
        run: npm ci

      - name: Run ESLint
        working-directory: frontend
        run: npm run lint

  test-backend:
    runs-on: ubuntu-latest
    name: Test Backend

    services:
      postgres:
        image: postgres:15-alpine
        env:
          POSTGRES_DB: test_db
          POSTGRES_USER: test_user
          POSTGRES_PASSWORD: test_password
        options: >-
          --health-cmd pg_isready
          --health-interval 10s
          --health-timeout 5s
          --health-retries 5
        ports:
          - 5432:5432

      redis:
        image: redis:7-alpine
        options: >-
          --health-cmd "redis-cli ping"
          --health-interval 10s
          --health-timeout 5s
          --health-retries 5
        ports:
          - 6379:6379

    steps:
      - uses: actions/checkout@v3

      - name: Set up Python
        uses: actions/setup-python@v4
        with:
          python-version: '3.11'
          cache: 'pip'
```

```yaml
      - name: Install dependencies
        run: |
          pip install -r backend/requirements.txt
          pip install pytest pytest-cov pytest-django

      - name: Run migrations
        env:
          DATABASE_URL:
              postgresql://test_user:test_password@localhost:5432/test_db
          DJANGO_SETTINGS_MODULE: synapse.settings.testing
        run: |
          cd backend
          python manage.py migrate

      - name: Run tests
        env:
          DATABASE_URL:
              postgresql://test_user:test_password@localhost:5432/test_db
          DJANGO_SETTINGS_MODULE: synapse.settings.testing
          REDIS_URL: redis://localhost:6379/0
        run: |
          cd backend
          pytest --cov=synapse --cov-report=xml --cov-report=html

      - name: Upload coverage reports
        uses: codecov/codecov-action@v3
        with:
          file: ./backend/coverage.xml
          flags: backend

  test-frontend:
    runs-on: ubuntu-latest
    name: Test Frontend

    steps:
      - uses: actions/checkout@v3

      - name: Set up Node.js
        uses: actions/setup-node@v3
        with:
          node-version: '20'
          cache: 'npm'
          cache-dependency-path: frontend/package-lock.json

      - name: Install dependencies
        working-directory: frontend
        run: npm ci

      - name: Run tests
        working-directory: frontend
        run: npm run test -- --coverage

      - name: Upload coverage reports
        uses: codecov/codecov-action@v3
        with:
          file: ./frontend/coverage/coverage-final.json
          flags: frontend
```

```yaml
  security-scan:
    runs-on: ubuntu-latest
    name: Security Scanning

    steps:
      - uses: actions/checkout@v3

      - name: Set up Python
        uses: actions/setup-python@v4
        with:
          python-version: '3.11'

      - name: Install security tools
        run: pip install bandit safety

      - name: Run Bandit (Python security)
        run: bandit -r backend/ -f json -o bandit-report.json || true

      - name: Run Safety (dependency check)
        run: safety check --json || true

      - name: Set up Node.js
        uses: actions/setup-node@v3
        with:
          node-version: '20'
          cache: 'npm'
          cache-dependency-path: frontend/package-lock.json

      - name: Install dependencies
        working-directory: frontend
        run: npm ci

      - name: Run npm audit
        working-directory: frontend
        run: npm audit || true

  build:
    needs: [lint, test-backend, test-frontend, security-scan]
    runs-on: ubuntu-latest
    name: Build Docker Images
    permissions:
      contents: read
      packages: write

    steps:
      - uses: actions/checkout@v3

      - name: Set up Docker Buildx
        uses: docker/setup-buildx-action@v2

      - name: Log in to Container Registry
        uses: docker/login-action@v2
        with:
          registry: ${{ env.REGISTRY }}
          username: ${{ github.actor }}
          password: ${{ secrets.GITHUB_TOKEN }}

      - name: Build and push backend image
        uses: docker/build-push-action@v4
```

```yaml
        with:
          context: ./backend
          push: ${{ github.event_name != 'pull_request' }}
          tags: |
            ${{ env.REGISTRY }}/${{ env.IMAGE_NAME }}/backend:${{ github.sha }}
            ${{ env.REGISTRY }}/${{ env.IMAGE_NAME }}/backend:latest
          cache-from: type=gha
          cache-to: type=gha,mode=max

      - name: Build and push fastapi image
        uses: docker/build-push-action@v4
        with:
          context: ./fastapi_ai
          push: ${{ github.event_name != 'pull_request' }}
          tags: |
            ${{ env.REGISTRY }}/${{ env.IMAGE_NAME }}/fastapi-ai:${{ github.sha }}
            ${{ env.REGISTRY }}/${{ env.IMAGE_NAME }}/fastapi-ai:latest
          cache-from: type=gha
          cache-to: type=gha,mode=max

      - name: Build and push frontend image
        uses: docker/build-push-action@v4
        with:
          context: ./frontend
          push: ${{ github.event_name != 'pull_request' }}
          tags: |
            ${{ env.REGISTRY }}/${{ env.IMAGE_NAME }}/frontend:${{ github.sha }}
            ${{ env.REGISTRY }}/${{ env.IMAGE_NAME }}/frontend:latest
          cache-from: type=gha
          cache-to: type=gha,mode=max
```

Listing 5.1: .github/workflows/ci.yml - Continuous Integration Pipeline

5.3 Continuous Deployment Workflow

```yaml
name: Deploy

on:
  push:
    branches: [ main ]

env:
  REGISTRY: ghcr.io
  IMAGE_NAME: ${{ github.repository }}
  DEPLOY_HOST: ${{ secrets.DEPLOY_HOST }}
  DEPLOY_USER: ${{ secrets.DEPLOY_USER }}
  DEPLOY_KEY: ${{ secrets.DEPLOY_KEY }}

jobs:
  build-and-push:
    runs-on: ubuntu-latest
    name: Build and Push Images
    permissions:
      contents: read
      packages: write
    outputs:
      image-tag: ${{ steps.meta.outputs.tags }}
```

```yaml
    steps:
      - uses: actions/checkout@v3

      - name: Set up Docker Buildx
        uses: docker/setup-buildx-action@v2

      - name: Log in to Container Registry
        uses: docker/login-action@v2
        with:
          registry: ${{ env.REGISTRY }}
          username: ${{ github.actor }}
          password: ${{ secrets.GITHUB_TOKEN }}

      - name: Build and push all images
        run: |
          docker buildx build --push -t ${{ env.REGISTRY }}/${{ env.IMAGE_NAME
              }}/backend:${{ github.sha }} ./backend
          docker buildx build --push -t ${{ env.REGISTRY }}/${{ env.IMAGE_NAME
              }}/fastapi-ai:${{ github.sha }} ./fastapi_ai
          docker buildx build --push -t ${{ env.REGISTRY }}/${{ env.IMAGE_NAME
              }}/frontend:${{ github.sha }} ./frontend

  deploy:
    needs: build-and-push
    runs-on: ubuntu-latest
    name: Deploy to Production

    steps:
      - uses: actions/checkout@v3

      - name: Deploy via SSH
        uses: appleboy/ssh-action@master
        with:
          host: ${{ env.DEPLOY_HOST }}
          username: ${{ env.DEPLOY_USER }}
          key: ${{ env.DEPLOY_KEY }}
          script: |
            cd /var/synapse
            echo "IMAGE_TAG=${{ github.sha }}" > .env.deploy
            source .env.deploy
            docker-compose pull
            docker-compose up -d

  health-check:
    needs: deploy
    runs-on: ubuntu-latest
    name: Health Check

    steps:
      - name: Check backend health
        run: |
          for i in {1..30}; do
            if curl -f https://api.synapse.example.com/health/; then
              echo "Backend is healthy"
              exit 0
            fi
            echo "Attempt $i failed, retrying..."
            sleep 10
```

```
          done
          echo "Backend health check failed"
          exit 1

      - name: Check frontend health
        run: |
          for i in {1..30}; do
            if curl -f https://synapse.example.com/api/health; then
              echo "Frontend is healthy"
              exit 0
            fi
            echo "Attempt $i failed, retrying..."
            sleep 10
          done
          echo "Frontend health check failed"
          exit 1

  notify:
    needs: [build-and-push, deploy, health-check]
    runs-on: ubuntu-latest
    name: Notify Deployment Status
    if: always()

    steps:
      - name: Notify Slack on success
        if: needs.health-check.result == 'success'
        uses: slackapi/slack-github-action@v1.24.0
        with:
          webhook-url: ${{ secrets.SLACK_WEBHOOK }}
          payload: |
            {
              "text": "SYNAPSE Deployment Successful",
              "blocks": [
                {
                  "type": "section",
                  "text": {
                    "type": "mrkdwn",
                    "text": "*SYNAPSE Deployment Successful* [OK]\n*Commit:* <${{
                        github.server_url }}/${{ github.repository }}/commit/${{
                        github.sha }}|${{ github.sha }}>\n*Author:* ${{
                        github.actor }}"
                  }
                }
              ]
            }

      - name: Notify Slack on failure
        if: needs.health-check.result == 'failure'
        uses: slackapi/slack-github-action@v1.24.0
        with:
          webhook-url: ${{ secrets.SLACK_WEBHOOK }}
          payload: |
            {
              "text": "SYNAPSE Deployment Failed",
              "blocks": [
                {
                  "type": "section",
                  "text": {
                    "type": "mrkdwn",
```

```
                    "text": "*SYNAPSE Deployment Failed* [FAIL]\n*Commit:* <${{
                        github.server_url }}/${{ github.repository }}/commit/${{
                        github.sha }}|${{ github.sha }}>\n*Author:* ${{
                        github.actor }}"
                }
            }
        ]
    }
```

Listing 5.2: .github/workflows/deploy.yml - Continuous Deployment Pipeline

5.4 GitHub Actions Best Practices

5.4.1 Secrets Management

All sensitive information is stored as GitHub Secrets:

- `DEPLOY_HOST`: Production server IP/hostname
- `DEPLOY_USER`: SSH user for deployment
- `DEPLOY_KEY`: SSH private key
- `SLACK_WEBHOOK`: Slack webhook URL for notifications
- `DATABASE_PASSWORD`: Production database password
- `API_KEYS`: Third-party API keys

5.4.2 Caching Strategy

The pipeline uses multiple caching layers to optimize build times:

- **pip cache**: Python dependencies cached between runs
- **npm cache**: Node.js dependencies cached
- **Docker layer cache**: Build layers cached via GitHub Actions cache
- **BuildKit inline cache**: Embedded in Docker images for efficiency

Chapter 6

Production Infrastructure (AWS)

6.1 Architecture Overview

SYNAPSE production infrastructure is deployed on AWS with high availability, auto-scaling, and disaster recovery capabilities:

1. **Application Load Balancer (ALB)**: Distributes traffic to EC2 instances
2. **EC2 Auto Scaling Group**: Manages backend and AI service instances
3. **RDS PostgreSQL**: Managed database with Multi-AZ for high availability
4. **ElastiCache Redis**: Managed in-memory cache cluster
5. **S3**: Object storage for media, backups, and logs
6. **CloudFront**: CDN for frontend static assets and API caching
7. **Route 53**: DNS management and health checks
8. **VPC**: Private subnets for databases, public subnets for ALB
9. **Security Groups**: Network-level access control
10. **CloudWatch**: Monitoring, logging, and alarms

6.2 EC2 Instance Recommendations

6.2.1 Backend Service

- **Instance Type**: t3.medium (2 vCPU, 4 GB RAM) - minimum
- **Recommended**: t3.large for production (2 vCPU, 8 GB RAM)
- **Volume**: 50 GB EBS (gp3) for application logs
- **Auto Scaling**: Min 2, Max 10 instances based on CPU ¿70%
- **Security Group**: Allow 8000 from ALB, SSH from bastion

6.2.2 FastAPI AI Service

- **Instance Type**: t3.large (2 vCPU, 8 GB RAM) - minimum
- **Recommended**: t3.xlarge (4 vCPU, 16 GB RAM) for AI workloads
- **Volume**: 100 GB EBS (gp3) for model caching
- **Auto Scaling**: Min 1, Max 5 instances based on queue depth
- **Security Group**: Allow 8001 from ALB, SSH from bastion

6.3 RDS PostgreSQL Configuration

```
Engine Version: PostgreSQL 15.x
Instance Class: db.t3.large (production minimum)
Multi-AZ: Enabled for high availability
Storage Type: gp3 (provisioned IOPS)
Storage Size: 100 GB (auto-scaling to 1000 GB)
Backup Retention: 30 days
Backup Window: 03:00-04:00 UTC
Maintenance Window: sun:04:00-sun:05:00 UTC
Enhanced Monitoring: Enabled (60 second granularity)
Performance Insights: Enabled (7 day retention)
Encryption: KMS encryption at rest
Network: Private subnet, accessible only from app tier
Database Name: synapse_prod
Master Username: synapse_admin
Parameter Groups:
  - max_connections: 100
  - shared_buffers: 256MB
  - effective_cache_size: 2GB
  - work_mem: 8MB
  - maintenance_work_mem: 64MB
```

Listing 6.1: RDS PostgreSQL Setup Parameters

6.4 ElastiCache Redis Configuration

```
Engine: Redis 7.x
Node Type: cache.t3.medium (production minimum)
Number of Nodes: 2 (Multi-AZ cluster mode disabled)
Parameter Group: custom (optimize for Celery)
Port: 6379
Encryption at Rest: Enabled (KMS)
Encryption in Transit: Enabled (TLS)
Automatic Failover: Enabled
Backup Retention: 5 days
Backup Window: 03:00-05:00 UTC
Maintenance Window: sun:04:00-sun:05:00 UTC
Parameter Overrides:
  - maxmemory-policy: allkeys-lru
  - timeout: 300
  - tcp-backlog: 511
```

Listing 6.2: ElastiCache Redis Setup Parameters

6.5 Auto-Scaling Configuration

```
{
  "AutoScalingGroupName": "synapse-backend-asg",
  "MinSize": 2,
  "MaxSize": 10,
  "DesiredCapacity": 2,
  "HealthCheckType": "ELB",
  "HealthCheckGracePeriod": 300,
  "TargetGroupArns": ["arn:aws:elasticloadbalancing:..."],
  "TargetTrackingScalingPolicy": {
    "TargetValue": 70.0,
    "PredefinedMetricSpecification": {
      "PredefinedMetricType": "ASGAverageCPUUtilization"
    },
    "ScaleOutCooldown": 60,
    "ScaleInCooldown": 300
  }
}
```

Listing 6.3: Auto-Scaling Policy - Backend

Chapter 7

Nginx Configuration

7.1 Reverse Proxy Setup

```
user nginx;
worker_processes auto;
error_log /var/log/nginx/error.log warn;
pid /var/run/nginx.pid;

events {
    worker_connections 2048;
    use epoll;
}

http {
    include /etc/nginx/mime.types;
    default_type application/octet-stream;

    # Logging
    log_format main '$remote_addr - $remote_user [$time_local] "$request" '
                    '$status $body_bytes_sent "$http_referer" '
                    '"$http_user_agent" "$http_x_forwarded_for"';
    access_log /var/log/nginx/access.log main;

    # Performance
    sendfile on;
    tcp_nopush on;
    tcp_nodelay on;
    keepalive_timeout 65;
    types_hash_max_size 2048;
    client_max_body_size 50M;

    # Gzip compression
    gzip on;
    gzip_vary on;
    gzip_proxied any;
    gzip_comp_level 6;
    gzip_types text/plain text/css text/xml text/javascript
               application/json application/javascript application/xml+rss
               application/rss+xml font/truetype font/opentype
               application/vnd.ms-fontobject image/svg+xml;

    # Rate limiting
    limit_req_zone $binary_remote_addr zone=api_limit:10m rate=10r/s;
```

```
    limit_req_zone $binary_remote_addr zone=general_limit:10m rate=30r/s;

    # Upstream definitions
    upstream django_backend {
        least_conn;
        server backend-1:8000 max_fails=3 fail_timeout=30s;
        server backend-2:8000 max_fails=3 fail_timeout=30s;
        server backend-3:8000 max_fails=3 fail_timeout=30s;
        keepalive 32;
    }

    upstream fastapi_backend {
        least_conn;
        server fastapi-ai-1:8001 max_fails=3 fail_timeout=30s;
        server fastapi-ai-2:8001 max_fails=3 fail_timeout=30s;
        keepalive 32;
    }

    upstream nextjs_frontend {
        least_conn;
        server frontend-1:3000 max_fails=3 fail_timeout=30s;
        server frontend-2:3000 max_fails=3 fail_timeout=30s;
        keepalive 32;
    }

    # HTTPS redirect
    server {
        listen 80;
        server_name synapse.example.com api.synapse.example.com
            ai.synapse.example.com;
        return 301 https://$server_name$request_uri;
    }

    # API Server
    server {
        listen 443 ssl http2;
        server_name api.synapse.example.com;

        # SSL certificates (Let's Encrypt)
        ssl_certificate
            /etc/letsencrypt/live/api.synapse.example.com/fullchain.pem;
        ssl_certificate_key
            /etc/letsencrypt/live/api.synapse.example.com/privkey.pem;
        ssl_protocols TLSv1.2 TLSv1.3;
        ssl_ciphers HIGH:!aNULL:!MD5;
        ssl_prefer_server_ciphers on;

        # Security headers
        add_header Strict-Transport-Security "max-age=31536000;
            includeSubDomains; preload" always;
        add_header X-Frame-Options "SAMEORIGIN" always;
        add_header X-Content-Type-Options "nosniff" always;
        add_header X-XSS-Protection "1; mode=block" always;
        add_header Referrer-Policy "no-referrer-when-downgrade" always;
        add_header Content-Security-Policy "default-src 'self'; script-src 'self'
            'unsafe-inline';" always;

        # Rate limiting
        limit_req zone=api_limit burst=20 nodelay;
```

```
        # Django proxy
        location / {
            proxy_pass http://django_backend;
            proxy_http_version 1.1;
            proxy_set_header Connection "";
            proxy_set_header Host $host;
            proxy_set_header X-Real-IP $remote_addr;
            proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
            proxy_set_header X-Forwarded-Proto $scheme;
            proxy_set_header X-Forwarded-Host $server_name;

            proxy_connect_timeout 30s;
            proxy_send_timeout 30s;
            proxy_read_timeout 30s;
        }

        # WebSocket support
        location /ws {
            proxy_pass http://django_backend;
            proxy_http_version 1.1;
            proxy_set_header Upgrade $http_upgrade;
            proxy_set_header Connection "upgrade";
            proxy_set_header Host $host;
            proxy_set_header X-Real-IP $remote_addr;
            proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;

            proxy_read_timeout 86400;
        }

        # Static files
        location /static/ {
            alias /var/www/synapse/static/;
            expires 30d;
            add_header Cache-Control "public, immutable";
        }

        # Media files
        location /media/ {
            alias /var/www/synapse/media/;
            expires 7d;
            add_header Cache-Control "public";
        }
    }

    # AI Service Server
    server {
        listen 443 ssl http2;
        server_name ai.synapse.example.com;

        ssl_certificate
            /etc/letsencrypt/live/ai.synapse.example.com/fullchain.pem;
        ssl_certificate_key
            /etc/letsencrypt/live/ai.synapse.example.com/privkey.pem;
        ssl_protocols TLSv1.2 TLSv1.3;

        add_header Strict-Transport-Security "max-age=31536000" always;

        location / {
```

```
            proxy_pass http://fastapi_backend;
            proxy_http_version 1.1;
            proxy_set_header Connection "";
            proxy_set_header Host $host;
            proxy_set_header X-Real-IP $remote_addr;
            proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
            proxy_set_header X-Forwarded-Proto $scheme;

            proxy_connect_timeout 60s;
            proxy_send_timeout 60s;
            proxy_read_timeout 60s;
        }
    }

    # Frontend Server
    server {
        listen 443 ssl http2;
        server_name synapse.example.com;

        ssl_certificate /etc/letsencrypt/live/synapse.example.com/fullchain.pem;
        ssl_certificate_key /etc/letsencrypt/live/synapse.example.com/privkey.pem;
        ssl_protocols TLSv1.2 TLSv1.3;

        add_header Strict-Transport-Security "max-age=31536000;
            includeSubDomains; preload" always;

        limit_req zone=general_limit burst=50 nodelay;

        # Next.js application
        location / {
            proxy_pass http://nextjs_frontend;
            proxy_http_version 1.1;
            proxy_set_header Connection "";
            proxy_set_header Host $host;
            proxy_set_header X-Real-IP $remote_addr;
            proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
            proxy_set_header X-Forwarded-Proto $scheme;
        }

        # Static assets with caching
        location /_next/static {
            proxy_pass http://nextjs_frontend;
            proxy_cache_valid 200 30d;
            add_header Cache-Control "public, immutable";
        }
    }
}
```

Listing 7.1: nginx.conf - Main Configuration

Chapter 8

Environment Configuration

8.1 Required Environment Variables

All production environment variables are managed through AWS Secrets Manager or HashiCorp Vault:

```
# Django/Django REST Framework
DJANGO_SECRET_KEY=<random-secret-key-min-50-chars>
DEBUG=False
ALLOWED_HOSTS=synapse.example.com,api.synapse.example.com,ai.synapse.example.com
CSRF_TRUSTED_ORIGINS=https://synapse.example.com,https://api.synapse.example.com

# Database
DATABASE_URL=postgresql://synapse_admin:password@rds-endpoint:5432/synapse_prod
DB_ECHO=False
DB_POOL_SIZE=20
DB_POOL_RECYCLE=3600

# Redis/Cache
REDIS_URL=redis://:password@elasticache-endpoint:6379/0
CACHE_BACKEND=django.core.cache.backends.redis.RedisCache

# Celery
CELERY_BROKER_URL=redis://:password@elasticache-endpoint:6379/1
CELERY_RESULT_BACKEND=redis://:password@elasticache-endpoint:6379/2
CELERY_TASK_SERIALIZER=json
CELERY_ACCEPT_CONTENT=json
CELERY_RESULT_SERIALIZER=json
CELERY_TIMEZONE=UTC

# AI/ML Services
OPENAI_API_KEY=<openai-secret-key>
OPENAI_ORG_ID=<optional-org-id>
PINECONE_API_KEY=<pinecone-api-key>
PINECONE_ENV=production
PINECONE_INDEX=synapse-prod

# External APIs
GITHUB_TOKEN=<github-personal-access-token>
GITHUB_API_BASE=https://api.github.com
ARXIV_API_KEY=<arxiv-api-key>
YOUTUBE_API_KEY=<youtube-api-key>
NEWS_API_KEY=<news-api-key>
NEWSAPI_BASE_URL=https://newsapi.org/v2
```

```
# AWS/Cloud
AWS_ACCESS_KEY_ID=<aws-access-key>
AWS_SECRET_ACCESS_KEY=<aws-secret-key>
AWS_STORAGE_BUCKET_NAME=synapse-prod-storage
AWS_S3_REGION_NAME=us-east-1
AWS_S3_CUSTOM_DOMAIN=cdn.synapse.example.com
AWS_DEFAULT_ACL=public-read
AWS_LOCATION=static

# Email/Notifications
SENDGRID_API_KEY=<sendgrid-api-key>
EMAIL_FROM=noreply@synapse.example.com
EMAIL_BACKEND=sendgrid_backend.SendgridBackend

# Monitoring
SENTRY_DSN=<sentry-dsn>
SENTRY_ENVIRONMENT=production
SENTRY_TRACES_SAMPLE_RATE=0.1

# Security
SECURE_SSL_REDIRECT=True
SESSION_COOKIE_SECURE=True
CSRF_COOKIE_SECURE=True
SECURE_HSTS_SECONDS=31536000
SECURE_HSTS_INCLUDE_SUBDOMAINS=True
SECURE_HSTS_PRELOAD=True

# Frontend
NEXT_PUBLIC_API_URL=https://api.synapse.example.com
NEXT_PUBLIC_FASTAPI_URL=https://ai.synapse.example.com
NEXT_PUBLIC_WS_URL=wss://api.synapse.example.com/ws
NEXT_PUBLIC_SENTRY_DSN=<frontend-sentry-dsn>
NEXT_PUBLIC_ENVIRONMENT=production
```

Listing 8.1: Complete Environment Variables Reference

8.2 Secrets Management Strategy

8.2.1 AWS Secrets Manager

```
{
  "DJANGO_SECRET_KEY": "value",
  "DATABASE_URL": "postgresql://user:pass@host/db",
  "OPENAI_API_KEY": "sk-...",
  "SENDGRID_API_KEY": "SG...",
  "AWS_ACCESS_KEY_ID": "AKIA...",
  "AWS_SECRET_ACCESS_KEY": "..."
}
```

Listing 8.2: AWS Secrets Manager Secret Structure

8.2.2 EC2 IAM Role

EC2 instances are granted an IAM role with permissions to:

- Read from Secrets Manager (GetSecretValue)

- Access S3 buckets (s3:GetObject, s3:PutObject)
- Write CloudWatch logs
- Access ElastiCache (conditional on security groups)
- Access RDS (conditional on security groups)

8.2.3 Deployment Secret Injection

During deployment, a startup script retrieves secrets from AWS Secrets Manager and injects them as environment variables before starting the application.

Chapter 9

Database Management in Production

9.1 Database Migrations Strategy

Zero-downtime database migrations are critical for production deployments:

1. **Backward-compatible changes**: All new columns added with default values or nullable
2. **Dual-write period**: Code writes to both old and new schemas
3. **Data migration**: Backfill data during low-traffic periods
4. **Code deployment**: Deploy code that reads from new columns
5. **Cleanup**: Remove old columns after verification
6. **Rollback plan**: Keep previous code version ready for quick rollback

```
1. Create new column/table with default values (backwards compatible)
2. Deploy code to write to both old and new locations
3. Wait for read replicas to catch up
4. Run migration job to backfill data from old to new
5. Deploy code to read from new location with fallback to old
6. Monitor error rates and verify data consistency
7. Remove writes to old location from code
8. Deploy updated code
9. Wait for all deployments
10. Drop old column/table in maintenance window
```

Listing 9.1: Database Migration Checklist

9.2 Backup Strategy

9.2.1 Automated Daily Backups

- **Frequency**: Daily automated backups at 03:00 UTC
- **Retention**: 30-day rolling backup window
- **Location**: AWS S3 with cross-region replication
- **Encryption**: KMS encryption at rest
- **Verification**: Weekly restore test to staging environment

9.2.2 Point-in-Time Recovery

RDS enables point-in-time recovery (PITR) within the backup retention window:

- **Recovery Window**: Any point in last 30 days
- **RTO**: 15-30 minutes depending on database size
- **Procedure**: Click *Actions ¿ Restore to Point in Time* in AWS console
- **Testing**: Monthly PITR drills to staging environment

9.3 Read Replicas for Reporting

Create read replicas to offload reporting queries from the primary database:

```
Primary Database: synapse-prod (us-east-1a)
Read Replica 1: synapse-prod-read-1 (us-east-1b) - Analytics
Read Replica 2: synapse-prod-read-2 (us-east-1c) - Backups
Lag Target: < 1 second
Connection Limit: 50 per replica
```

Listing 9.2: Read Replica Configuration

Reporting queries are routed to read replicas using Django database router.

9.4 pgvector Index Maintenance

9.4.1 Index Creation

For optimal AI search performance, create indices on vector columns:

```
-- Create HNSW index for vector similarity search
CREATE INDEX ON embeddings USING hnsw (embedding vector_cosine_ops)
WITH (m=16, ef_construction=200);

-- Create IVFFlat index (alternative for larger datasets)
CREATE INDEX ON embeddings USING ivfflat (embedding vector_cosine_ops)
WITH (lists=100);

-- Create B-tree index for metadata filtering
CREATE INDEX ON embeddings (created_at DESC) WHERE deleted_at IS NULL;

-- Analyze distribution
ANALYZE embeddings;
```

Listing 9.3: pgvector Index Creation

9.4.2 Index Maintenance

```
#!/bin/bash
# Run weekly to maintain vector indices

psql $DATABASE_URL << EOF
-- Reindex regularly for optimal performance
REINDEX INDEX CONCURRENTLY embeddings_embedding_idx;

```

```
-- Vacuum to recover space from deleted embeddings
VACUUM ANALYZE embeddings;

-- Check index bloat
SELECT
  schemaname, tablename, indexname,
  ROUND(100 * (OTTA - CASE WHEN OTTA=0 THEN 1 ELSE OTTA END) / OTTA::numeric) AS
      n_dead_tup
FROM pgstattuple_approx('embeddings_embedding_idx'::regclass)
WHERE OTTA > 0;
EOF
```

Listing 9.4: pgvector Index Maintenance Script

9.5 Redis Persistence

Redis uses both AOF (Append-Only File) and RDB (snapshot) for durability:

```
# RDB Snapshots (point-in-time backup)
save 900 1        # Save if 1 key changed in 900 seconds
save 300 10       # Save if 10 keys changed in 300 seconds
save 60 10000     # Save if 10000 keys changed in 60 seconds

# AOF (rewrite every operation for durability)
appendonly yes
appendfsync everysec
auto-aof-rewrite-percentage 100
auto-aof-rewrite-min-size 64mb

# Disk space limits
maxmemory-policy allkeys-lru
maxmemory 2gb
```

Listing 9.5: Redis Persistence Configuration

Chapter 10

Monitoring & Observability

10.1 Prometheus Metrics Setup

10.1.1 Django Metrics with django-prometheus

```
# settings.py
INSTALLED_APPS = [
    'django_prometheus',
    # ... other apps
]

MIDDLEWARE = [
    'django_prometheus.middleware.PrometheusBeforeMiddleware',
    # ... other middleware
    'django_prometheus.middleware.PrometheusAfterMiddleware',
]

# urls.py
from django_prometheus import urls as prometheus_urls

urlpatterns = [
    path('', include(prometheus_urls)),
    # ... other patterns
]
```

Listing 10.1: Django Prometheus Configuration

10.1.2 Custom Business Metrics

```
from prometheus_client import Counter, Gauge, Histogram

# Articles scraped
articles_scraped = Counter(
    'articles_scraped_total',
    'Total articles scraped',
    ['source']
)

# AI requests
ai_requests = Counter(
    'ai_requests_total',
```

```
    'Total AI API requests',
    ['model', 'status']
)

# Agent tasks completed
agent_tasks = Counter(
    'agent_tasks_completed_total',
    'Total agent tasks completed',
    ['task_type', 'status']
)

# Queue depth
celery_queue_depth = Gauge(
    'celery_queue_depth',
    'Current Celery task queue depth',
    ['queue_name']
)

# Request latency
request_latency = Histogram(
    'request_latency_seconds',
    'API request latency',
    ['endpoint'],
    buckets=(0.1, 0.5, 1.0, 2.0, 5.0)
)
```

Listing 10.2: Custom Prometheus Metrics

10.2 Grafana Dashboards

10.2.1 System Overview Dashboard

Dashboard displays:

- CPU usage across instances
- Memory utilization per service
- Disk I/O and storage usage
- Network throughput
- Container restart counts

10.2.2 Application Metrics Dashboard

- Request rate (req/s)
- Response time (p50, p95, p99)
- Error rate by endpoint
- Status code distribution
- Database connection pool usage

10.2.3 Business Metrics Dashboard

- Daily articles scraped (by source)
- AI API request volume
- Agent task completion rate
- Feature usage by user
- Customer acquisition funnel

10.2.4 Celery Task Monitoring

- Task queue depth
- Task processing rate
- Task failure rate
- Worker availability
- Task execution time (p50, p95, p99)

10.3 Sentry Integration

10.3.1 Django Setup

```python
import sentry_sdk
from sentry_sdk.integrations.django import DjangoIntegration
from sentry_sdk.integrations.celery import CeleryIntegration

sentry_sdk.init(
    dsn=os.getenv('SENTRY_DSN'),
    integrations=[
        DjangoIntegration(),
        CeleryIntegration(),
    ],
    traces_sample_rate=0.1,
    send_default_pii=False,
    environment=os.getenv('SENTRY_ENVIRONMENT', 'development'),
)
```

Listing 10.3: Sentry Configuration - Django

10.3.2 Next.js Setup

```javascript
// next.config.js
const withSentryConfig = require("@sentry/nextjs/config");

module.exports = withSentryConfig({
  // ... other config
}, {
  org: process.env.SENTRY_ORG,
  project: process.env.SENTRY_PROJECT,
  authToken: process.env.SENTRY_AUTH_TOKEN,
});

```

```
// pages/_document.js
import * as Sentry from "@sentry/nextjs";

Sentry.init({
  dsn: process.env.NEXT_PUBLIC_SENTRY_DSN,
  environment: process.env.NEXT_PUBLIC_ENVIRONMENT,
  tracesSampleRate: 0.1,
});
```

Listing 10.4: Sentry Configuration - Next.js

10.4 Structured Logging

All logs are structured as JSON for easy aggregation and analysis:

```
LOGGING = {
    'version': 1,
    'disable_existing_loggers': False,
    'formatters': {
        'json': {
            '()': 'pythonjsonlogger.jsonlogger.JsonFormatter',
            'format': '%(asctime)s %(name)s %(levelname)s %(message)s'
        }
    },
    'handlers': {
        'console': {
            'class': 'logging.StreamHandler',
            'formatter': 'json',
        },
        'file': {
            'class': 'logging.handlers.RotatingFileHandler',
            'filename': '/var/log/synapse/django.log',
            'maxBytes': 1024 * 1024 * 100,  # 100 MB
            'backupCount': 10,
            'formatter': 'json',
        }
    },
    'root': {
        'handlers': ['console', 'file'],
        'level': 'INFO',
    }
}
```

Listing 10.5: JSON Logging Configuration

10.4.1 Log Aggregation with CloudWatch

CloudWatch Logs Insights queries for common scenarios:

```
# Error rate over time
fields @timestamp, @message, severity
| filter severity = "ERROR"
| stats count() by bin(5m)

# Slow requests
fields @timestamp, @message, duration
| filter duration > 1000
```

```
| sort duration desc
| limit 100

# Failed API calls
fields @timestamp, endpoint, status_code, error
| filter status_code >= 400
| stats count() by endpoint, status_code

# Celery task failures
fields @timestamp, task_name, exception
| filter severity = "ERROR" and source = "celery"
| stats count() by task_name
```

Listing 10.6: CloudWatch Logs Insights Queries

10.5 Uptime Monitoring

10.5.1 AWS Route 53 Health Checks

```
Type: HTTPS
Resource Path: /health/
Port: 443
Request Interval: 30 seconds
Failure Threshold: 3 consecutive failures
Regions: Virginia, California, Ireland
CloudWatch Alarm: Alert on 2+ regions failing
```

Listing 10.7: Health Check Configuration

10.6 Alert Rules

Production alerting thresholds:

```
# Error Rate Alert
Condition: error_rate > 5% for 5 minutes
Action: Page on-call engineer via PagerDuty
Severity: CRITICAL

# High CPU Alert
Condition: CPU utilization > 80% for 10 minutes
Action: Slack notification
Severity: WARNING

# Disk Usage Alert
Condition: Disk used > 90% for 5 minutes
Action: Slack notification + PagerDuty
Severity: CRITICAL

# Database Connection Pool Alert
Condition: Connection pool utilization > 80%
Action: Slack notification
Severity: WARNING

# Celery Queue Backlog Alert
Condition: Queue depth > 10000 tasks for 15 minutes
```

```
Action: Slack notification
Severity: WARNING

# Redis Memory Alert
Condition: Memory usage > 85% of max
Action: Slack notification
Severity: WARNING
```

Listing 10.8: Alert Rules and Thresholds

Chapter 11

Security Hardening

11.1 Django Security Settings

```
# settings/production.py

# SSL/HTTPS
SECURE_SSL_REDIRECT = True
SESSION_COOKIE_SECURE = True
CSRF_COOKIE_SECURE = True
SECURE_BROWSER_XSS_FILTER = True
SECURE_CONTENT_SECURITY_POLICY = {
    "default-src": ("'self'",),
    "script-src": ("'self'", "cdn.jsdelivr.net"),
    "style-src": ("'self'", "'unsafe-inline'", "fonts.googleapis.com"),
}

# HSTS (HTTP Strict Transport Security)
SECURE_HSTS_SECONDS = 31536000  # 1 year
SECURE_HSTS_INCLUDE_SUBDOMAINS = True
SECURE_HSTS_PRELOAD = True

# CORS
CORS_ALLOWED_ORIGINS = [
    "https://synapse.example.com",
    "https://api.synapse.example.com",
]

# Session security
SESSION_COOKIE_HTTPONLY = True
SESSION_COOKIE_SAMESITE = "Strict"
CSRF_COOKIE_HTTPONLY = True

# Password validation
AUTH_PASSWORD_VALIDATORS = [
    {
        'NAME': 'django.contrib.auth.password_validation'
                '.UserAttributeSimilarityValidator',
    },
    {
        'NAME': 'django.contrib.auth.password_validation'
                '.MinimumLengthValidator',
        'OPTIONS': {'min_length': 12}
    },
```

```
    {
        'NAME': 'django.contrib.auth.password_validation'
                '.CommonPasswordValidator',
    },
    {
        'NAME': 'django.contrib.auth.password_validation'
                '.NumericPasswordValidator',
    },
]
```

Listing 11.1: Production Django Security Configuration

11.2 Database Security

- **SSL Connections**: All database connections use SSL/TLS
- **Least Privilege**: Application user has minimal required permissions
- **No Public Access**: Database is in private subnet, not accessible from internet
- **Encryption at Rest**: KMS encryption enabled
- **Backup Encryption**: All backups encrypted with KMS

11.3 Docker Security

- **Non-root User**: All containers run as unprivileged user (UID 1000)
- **Read-only Filesystem**: Applications run with read-only root where possible
- **Resource Limits**: Memory and CPU limits enforced
- **Image Scanning**: Container images scanned for vulnerabilities
- **No Secrets in Images**: Secrets injected at runtime only

11.4 Network Security

```
# Backend Security Group
Ingress:
  - HTTP:8000 from ALB
  - SSH:22 from Bastion
Egress:
  - All traffic to RDS, Redis, S3

# Database Security Group
Ingress:
  - PostgreSQL:5432 from Backend SG
Egress:
  - Minimal (mostly disabled)

# Redis Security Group
Ingress:
  - Redis:6379 from Backend SG, Celery SG
Egress:
  - Minimal
```

Listing 11.2: AWS Security Group Configuration

11.5 Dependency Scanning

11.5.1 Python Dependencies

- **Tool**: Safety + Dependabot
- **Frequency**: Continuous (on every commit)
- **Action**: Block merge if HIGH/CRITICAL vulnerabilities
- **Update Policy**: Security patches applied within 24 hours

11.5.2 Node.js Dependencies

- **Tool**: npm audit + Dependabot
- **Frequency**: Continuous
- **Action**: Automated PRs for updates
- **Update Policy**: Security patches applied within 48 hours

11.6 Secret Rotation Strategy

1. **API Keys**: Rotate every 90 days
2. **Database Password**: Rotate every 180 days
3. **Django SECRET_KEY**: Rotate on security incident or annually
4. **SSL Certificates**: Auto-renewal 30 days before expiration
5. **AWS Keys**: Rotate every 180 days
6. **Procedure**: Rotation done in maintenance window with zero-downtime

Chapter 12

Scaling Strategy

12.1 Horizontal Scaling of Backend Services

Backend services scale horizontally using AWS Auto Scaling Groups:

1. **Target Tracking**: Scale based on CPU utilization (target 70%)
2. **Min Instances**: 2 for high availability
3. **Max Instances**: 10 to control costs
4. **Scale-up**: Add instance within 1 minute of exceeding threshold
5. **Scale-down**: Remove instance 5 minutes after dropping below threshold
6. **Termination Policy**: Terminate oldest instances first

12.2 Celery Worker Scaling

Auto-scale Celery workers based on queue depth:

```python
import boto3
from celery import current_app

def get_queue_depth():
    """Get current queue depth from Redis"""
    inspector = current_app.control.inspect()
    active_tasks = inspector.active()
    queue_depth = sum(len(tasks) for tasks in active_tasks.values())
    return queue_depth

def scale_workers(queue_depth):
    """Scale worker count based on queue depth"""
    asg_client = boto3.client('autoscaling')

    target_workers = min(10, max(2, queue_depth // 100))

    asg_client.set_desired_capacity(
        AutoScalingGroupName='synapse-celery-asg',
        DesiredCapacity=target_workers,
        HonorCooldown=False
    )
```

Listing 12.1: Queue Depth Based Scaling

12.3 Database Connection Pooling (PgBouncer)

PgBouncer reduces database connection overhead:

```
[databases]
synapse_prod = host=rds-endpoint port=5432 dbname=synapse_prod

[pgbouncer]
pool_mode = transaction
max_client_conn = 500
default_pool_size = 25
min_pool_size = 10
reserve_pool_size = 5
reserve_pool_timeout = 3
max_db_connections = 100
server_lifetime = 3600
```

Listing 12.2: PgBouncer Configuration

12.4 CDN for Static Assets

CloudFront CDN configuration for optimal static asset delivery:

1. **Origin**: S3 bucket for frontend static files
2. **Distribution**: Global (217 edge locations)
3. **Caching**: 30 days for immutable assets, 1 hour for HTML
4. **Compression**: Gzip and Brotli compression enabled
5. **Security**: TLS 1.2+, only HTTPS
6. **Cost**: Approximately $0.085 per GB transferred

12.5 Redis Cluster for High Availability

Redis Cluster setup for fault tolerance:

```
Cluster Mode: Enabled
Node Type: cache.r6g.xlarge
Number of Node Groups: 3
Replicas per Node Group: 2
Automatic Failover: Enabled
Multi-AZ: Enabled
Total Nodes: 9 (3 shards x 3 replicas each)
Capacity: 750 GB total (250 GB per shard)
```

Listing 12.3: Redis Cluster Configuration

12.6 Load Testing

Use Locust for production-like load testing before deployments:

```python
from locust import HttpUser, task, between

class SynapseUser(HttpUser):
    wait_time = between(1, 5)

    @task(3)
    def get_dashboard(self):
        self.client.get("/api/dashboard/")

    @task(2)
    def search_articles(self):
        self.client.get("/api/articles/?search=AI&limit=20")

    @task(1)
    def get_ai_insights(self):
        self.client.post("/api/ai/insights/",
            json={"topic": "machine learning"})

    def on_start(self):
        # Login
        response = self.client.post("/api/auth/login/",
            json={"username": "testuser", "password": "testpass"})
        self.client.headers.update({
            "Authorization": f"Bearer {response.json()['access_token']}"
        })
```

Listing 12.4: Locust Load Testing Script

Chapter 13

Disaster Recovery

13.1 Recovery Objectives

- **RTO (Recovery Time Objective)**: 1 hour to restore full functionality
- **RPO (Recovery Point Objective)**: 24 hours (point-in-time recovery)
- **Availability SLA**: 99.95% uptime (¡ 22 minutes downtime/month)

13.2 Multi-AZ Database Deployment

RDS Multi-AZ synchronous replication ensures high availability:

1. **Primary**: us-east-1a
2. **Standby**: us-east-1b (synchronous replica)
3. **Failover Time**: ¡ 2 minutes automatic failover
4. **No Data Loss**: Synchronous replication guarantees durability
5. **No Manual Intervention**: Automatic DNS update on failover

13.3 S3 Versioning and Cross-Region Replication

S3 bucket protection against accidental deletion:

```
Bucket: synapse-prod-storage
Versioning: Enabled (retain all versions)
MFA Delete: Enabled (require MFA to permanently delete)
Cross-Region Replication:
  - Replication to us-west-2 (secondary region)
  - Replication to eu-west-1 (tertiary region)
  - Encryption: KMS (separate key per region)
Lifecycle Policy:
  - Non-current versions: Delete after 90 days
  - Incomplete multipart uploads: Delete after 7 days
```

Listing 13.1: S3 Protection Strategy

13.4 Infrastructure Replication

Production infrastructure is replicated across regions:

1. **Primary Region**: us-east-1 (N. Virginia)
2. **Secondary Region**: us-west-2 (Oregon)
3. **Tertiary Region**: eu-west-1 (Ireland)
4. **DNS**: Route 53 geolocation routing + health checks
5. **Failover**: Automatic DNS update if primary unhealthy
6. **Data Sync**: Cross-region RDS read replicas
7. **Cost**: + 40% for secondary region (standby)

13.5 Disaster Recovery Runbook

13.5.1 Database Corruption

1. Stop application traffic (update Route 53 to return 503)
2. Identify corruption point using logs
3. Perform point-in-time recovery to 1 hour before corruption
4. Validate recovered data against backups
5. Test on staging environment first
6. Restore traffic
7. Post-mortem investigation

13.5.2 Complete Data Center Failure

1. Activate disaster recovery site (secondary region)
2. Route DNS to secondary region via Route 53
3. Promote read replicas to standalone databases
4. Verify data consistency and freshness
5. Update frontend configuration to point to secondary region
6. Test critical workflows
7. Restore from latest backup if data loss detected
8. RTO: ¡ 1 hour

13.5.3 Ransomware/Security Incident

1. Isolate affected systems from network
2. Disable all API keys and credentials
3. Take offline backups (snapshot current state)
4. Restore from clean backup (before infection detected)
5. Rotate all secrets and credentials
6. Enable enhanced monitoring and logging
7. Security audit of compromised systems
8. Deploy patched code
9. Restore traffic gradually

Chapter 14

Deployment Runbook

14.1 Pre-Deployment Checklist

Before any production deployment, verify:

- All tests passing (lint, unit, integration, e2e)
- Security scanning completed (no HIGH/CRITICAL vulnerabilities)
- Database migrations reviewed and tested on staging
- Feature flags configured for gradual rollout
- Monitoring dashboards verified and alerts configured
- On-call engineer notified and standing by
- Rollback procedure documented and tested
- Backup created before deployment
- Maintenance window scheduled if needed

14.2 Step-by-Step Deployment Procedure

1. **Trigger Deployment**: Push to main branch (GitHub Actions auto-deploys)
2. **Build Stage**: Docker images built and pushed to registry
3. **Deploy Stage**: SSH to production servers, pull images, start containers
4. **Health Check Stage**: Verify endpoints responding and databases accessible
5. **Smoke Test**: Run automated smoke tests against live endpoints
6. **Monitor Stage**: Watch error rates, latency, and database connections for 15 minutes
7. **Notify Stage**: Send Slack notification to team with deployment summary

14.2.1 Manual Deployment (if CI/CD unavailable)

```bash
#!/bin/bash
# Manual deployment script (use only in emergency)

set -e

# Configuration
DEPLOY_USER="deploy"
DEPLOY_HOST="prod-server.example.com"
IMAGE_REGISTRY="ghcr.io/yourorg/synapse"
IMAGE_TAG=$(git rev-parse --short HEAD)

# 1. Build images
docker build -t $IMAGE_REGISTRY/backend:$IMAGE_TAG ./backend
docker build -t $IMAGE_REGISTRY/fastapi-ai:$IMAGE_TAG ./fastapi_ai
docker build -t $IMAGE_REGISTRY/frontend:$IMAGE_TAG ./frontend

# 2. Push to registry
docker push $IMAGE_REGISTRY/backend:$IMAGE_TAG
docker push $IMAGE_REGISTRY/fastapi-ai:$IMAGE_TAG
docker push $IMAGE_REGISTRY/frontend:$IMAGE_TAG

# 3. SSH to production and deploy
ssh $DEPLOY_USER@$DEPLOY_HOST << EOF
    set -e
    cd /var/synapse

    # Pull latest images
    docker pull $IMAGE_REGISTRY/backend:$IMAGE_TAG
    docker pull $IMAGE_REGISTRY/fastapi-ai:$IMAGE_TAG
    docker pull $IMAGE_REGISTRY/frontend:$IMAGE_TAG

    # Run migrations
    docker-compose run --rm backend python manage.py migrate

    # Update docker-compose.yml with new image tag
    sed -i 's|image: .*backend.*|image: $IMAGE_REGISTRY/backend:$IMAGE_TAG|g'
        docker-compose.yml
    sed -i 's|image: .*fastapi.*|image: $IMAGE_REGISTRY/fastapi-ai:$IMAGE_TAG|g'
        docker-compose.yml
    sed -i 's|image: .*frontend.*|image: $IMAGE_REGISTRY/frontend:$IMAGE_TAG|g'
        docker-compose.yml

    # Start new containers (rolling update)
    docker-compose up -d --no-deps --scale backend=3 backend
    docker-compose up -d --no-deps fastapi_ai
    docker-compose up -d --no-deps frontend

    # Wait for health checks
    sleep 30

    # Verify deployment
    curl -f http://localhost:8000/health/ || exit 1
    curl -f http://localhost:8001/health || exit 1
EOF

echo "Deployment successful!"
```

Listing 14.1: Manual Deployment Steps

14.3 Rollback Procedure

If deployment causes issues, rollback quickly:

1. **Detect Issue**: Monitor alerts for error rate spike, increased latency, or failures
2. **Decide to Rollback**: If error rate ¿ 10%, rollback immediately
3. **Rollback Code**: Revert to previous commit via GitHub
4. **Rebuild Images**: GitHub Actions automatically rebuilds and deploys old version
5. **Verify**: Run health checks and smoke tests
6. **Post-Incident**: Investigate root cause, add tests, and retry deployment

```bash
#!/bin/bash
# Rollback to previous deployed version

DEPLOY_USER="deploy"
DEPLOY_HOST="prod-server.example.com"

ssh $DEPLOY_USER@$DEPLOY_HOST << 'EOF'
    cd /var/synapse

    # Stop current deployment
    docker-compose stop

    # Checkout previous version
    git checkout HEAD~1

    # Rebuild and restart with previous version
    docker-compose build
    docker-compose up -d

    # Run migrations (if needed)
    docker-compose exec -T backend python manage.py migrate

    # Verify
    sleep 10
    curl -f http://localhost:8000/health/ || exit 1

    echo "Rollback complete"
EOF
```

Listing 14.2: Manual Rollback Script

14.4 Health Check Endpoints

All services expose health check endpoints for monitoring:

```
# Django Backend
GET /health/
Response: {"status": "ok", "version": "1.0.0", "timestamp":
    "2024-01-15T10:30:00Z"}
Dependencies: PostgreSQL, Redis, Celery

# FastAPI AI Service
GET /health
Response: {"status": "ok", "version": "1.0.0", "gpu_available": true}
Dependencies: PostgreSQL, Pinecone, OpenAI API

# Next.js Frontend
GET /api/health
Response: {"status": "ok", "build_id": "abc123def456"}
Dependencies: Static assets available
```

Listing 14.3: Health Check Endpoints Reference

14.5 Post-Deployment Verification Checklist

After deployment, verify:

- All services responding to health checks
- Error rate ¡ 1% (normal operation)
- Response latency p95 ¡ 500ms (normal operation)
- Database replication lag ¡ 1 second
- Celery tasks processing normally (queue depth ¡ 100)
- Log ingestion normal (no errors in real-time logs)
- User-facing features working (smoke tests passed)
- Feature flags behaving as expected
- Load balancer routing traffic correctly
- SSL certificates valid and not expiring soon
- Monitoring alerts not firing (unless expected)

14.6 Maintenance Window Procedure

For database migrations or major changes requiring downtime:

1. Schedule 2-week advance notice to users
2. Set deployment mode to maintenance (return HTTP 503)
3. Notify team 1 hour before window
4. Perform database migrations with verification
5. Perform code deployment
6. Run post-deployment tests

7. Remove maintenance mode
8. Monitor for 30 minutes for any issues
9. Notify team of successful completion

Conclusion

This DevOps and Deployment guide provides a comprehensive framework for operating SYNAPSE in production. Key principles include:

- **Infrastructure as Code**: All infrastructure is versioned and reproducible
- **Automation First**: Minimize manual processes through CI/CD
- **Observability**: Comprehensive monitoring and logging for quick issue detection
- **Security by Default**: Multi-layer defense and regular security reviews
- **Reliability**: Multi-AZ, automated failover, and disaster recovery
- **Scalability**: Horizontal scaling and efficient resource utilization
- **Maintainability**: Clear runbooks and documented procedures

For updates to this document or to report issues, contact the DevOps team.

Appendix A

Useful Commands

A.1 Docker Compose Commands

```
# Start all services
docker-compose up -d

# View logs
docker-compose logs -f [service_name]

# Execute command in container
docker-compose exec backend python manage.py shell

# Stop services
docker-compose stop

# Remove containers and volumes
docker-compose down -v

# Scale service
docker-compose up -d --scale celery_worker=3

# View resource usage
docker stats
```

A.2 AWS CLI Commands

```
# Get RDS endpoint
aws rds describe-db-instances --db-instance-identifier synapse-prod \
  --query 'DBInstances[0].Endpoint.Address'

# Get ElastiCache endpoint
aws elasticache describe-cache-clusters \
  --cache-cluster-id synapse-redis \
  --show-cache-node-info

# Get secrets from Secrets Manager
aws secretsmanager get-secret-value \
  --secret-id synapse/prod

# View CloudWatch logs
```

```
aws logs tail /aws/synapse/django --follow

# Create RDS snapshot
aws rds create-db-snapshot \
  --db-instance-identifier synapse-prod \
  --db-snapshot-identifier synapse-backup-$(date +%s)
```

A.3 Database Commands

```
# Connect to RDS
psql postgresql://user:password@rds-endpoint:5432/synapse_db

# Useful queries in psql
-- View database size
SELECT pg_size_pretty(pg_database_size('synapse_db'));

-- View table sizes
SELECT relname, pg_size_pretty(pg_total_relation_size(relid))
FROM pg_catalog.pg_statio_user_tables
ORDER BY pg_total_relation_size(relid) DESC;

-- View active connections
SELECT count(*) FROM pg_stat_activity;

-- View slow queries (requires log_statement=all)
SELECT query, calls, mean_exec_time FROM pg_stat_statements
ORDER BY mean_exec_time DESC LIMIT 10;
```